

```
In [4]: # Initialize Otter
import otter
grader = otter.Notebook("HW3.ipynb")
```

Grading

The final score that you will receive for your programming assignment is generated in relation to the total points set in your programming assignment item—not the total point value in the autograder output.

Autograded questions may have hidden tests and/or public tests. You can see the public test results when you run the notebook with otter grader. Usually public tests are for simple checks (such as variable types) and may not indicate that the answer is correct.

DO NOT CHANGE VARIABLE OR METHOD SIGNATURES The autograder will not work properly if you change the variable or method signatures.

K-nearest neighbors

Run the cell below to ensure that the required packages are imported.

```
In [5]: import math
import pickle
import gzip
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

# importing all the required libraries

from math import exp
import numpy as np
import pandas as pd
import sklearn
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
%matplotlib inline

from sklearn.metrics import roc_auc_score
from sklearn.metrics import roc_curve
```

Problem 1 : Building a K- Nearest neighbours classifier for handwritten digit recognition [30 pts]

In this problem you will complete some code to build a k-nearest neighbour classifier to classify images of handwritten digits (0-9). For this purpose we will use a famous open-

source dataset of handwritten digits called the MNIST that is ubiquitously used for testing a number of classification algorithms in machine learning.

```
In [6]: # This cell sets up the MNIST dataset

class MNIST_import:
    """
    sets up MNIST dataset from OpenML
    """
    def __init__(self):

        df = pd.read_csv("data/mnist_784.csv.gz", compression='gzip')

        # Create arrays for the features and the response variable
        # store for use later
        y = df['class'].values
        X = df.drop('class', axis=1).values

        # Convert the labels to numeric labels
        y = np.array(pd.to_numeric(y))

        # create training and validation sets
        self.train_x, self.train_y = X[:5000,:], y[:5000]
        self.val_x, self.val_y = X[5000:6000,:], y[5000:6000]

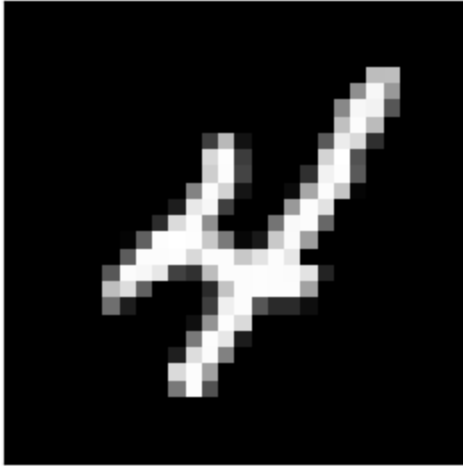
data = MNIST_import()
```

```
In [7]: def view_digit(x, label=None):
        fig = plt.figure(figsize=(3,3))
        plt.imshow(x.reshape(28,28), cmap='gray');
        plt.xticks([]); plt.yticks([]);
        if label: plt.xlabel("true: {}".format(label), fontsize=16)
```

Display a particular digit using the above function:

```
In [9]: training_index = 9

view_digit(data.train_x[training_index], data.train_y[training_index])
```



true: 4

Part 1.A [5 points] Fill in the code in the following cell to determine the following quantities:

- Number of pixels in each image
- Number of examples in the training set
- Number of examples in the test set

```
In [10]: # Here are the numbers you need to provide here:
num_training_examples = data.train_x.shape[0] # number of rows in training s
num_test_examples = data.val_x.shape[0] # number of rows in validation/test
pixels_per_image = data.train_x.shape[1]

...

print(num_training_examples)
print(num_test_examples)
print(pixels_per_image)
```

```
5000
1000
784
```

Now that we have our MNIST data in the right form, let us move on to building our KNN classifier.

Part 1.B [10 points]: Modify the class above to implement a KNN classifier. There are three methods that you need to complete:

- `predict` : Given an $m \times p$ matrix of validation data with m examples each with p features, return a length- m vector of predicted labels by calling the `classify` function on each example.
- `classify` : Given a single query example with p features, return its predicted class label as an integer using KNN by calling the `majority` function.
- `majority` : Given an array of indices into the training set corresponding to the K training examples that are nearest to the query point, return the majority label as an

integer. If there is a tie for the majority label using K nearest neighbors, reduce K by 1 and try again. Continue reducing K until there is a winning label.

Notes:

- Do not implement nearest-neighbor search or any distance metrics yourself. Instead, read the documentation for Scikit-Learn's [BallTree](#) object. You will find that its implemented [query](#) method can do most of the heavy lifting for you.
- Do not use Scikit-Learn's `KNeighborsClassifier` in this problem. We're implementing this ourselves using `BallTree`.
- For more information about different algorithms under the K-nearest neighbors and which algorithms are being used by sklearn in different scenarios, please see User guide p.263.

```
In [15]: class KNN:
        """
        Class to store data for regression problems
        """
        def __init__(self, x_train, y_train, K=5):
            """
            Creates a kNN instance

            :param x_train: numpy array with shape (n_rows,1)- e.g. [[1,2],[3,4]]
            :param y_train: numpy array with shape (n_rows,)- e.g. [1,-1]
            :param K: The number of nearest points to consider in classification
            """

            # Import and build the BallTree on training features
            from sklearn.neighbors import BallTree
            self.balltree = BallTree(x_train)

            # Cache training labels and parameter K
            self.y_train = y_train
            self.K = K

        def majority(self, neighbor_indices, neighbor_distances=None):
            """
            Given indices of nearest neighbors in training set, return the majority label.
            Break ties by considering 1 fewer neighbor until a clear winner is found.

            :param neighbor_indices: The indices of the K nearest neighbors in training set
            :param neighbor_distances: Corresponding distances from query point
            """

            neighbor_indices = np.array(neighbor_indices, dtype=int)
            k = len(neighbor_indices)
            while k > 0:
                top_k_indices = neighbor_indices[:k]
                labels, counts = np.unique(self.y_train[top_k_indices], return_counts=True)
                max_count = np.max(counts)
```

```

        # check if there's a tie
        if np.sum(counts == max_count) == 1:
            return labels[np.argmax(counts)]
        else:
            k -= 1 # reduce k and try again
    # if k reduces to 0, return label of nearest neighbor
    return self.y_train[neighbor_indices[0]]

def classify(self, x):
    """
    Given a query point, return the predicted label

    :param x: a query point stored as an ndarray
    """
    dist, ind = self.balltree.query([x], k=self.K)
    return self.majority(ind[0], dist[0])

def predict(self, X):
    """
    Given an ndarray of query points, return yhat, an ndarray of predicted labels

    :param X: an (m x p) dimension ndarray of points to predict labels for
    """
    return np.array([self.classify(x) for x in X])

```

Part 1.C : Checking how well your classifier does [5 pts]

Use your `KNN` class to perform KNN on the validation data with $K = 3$ and do the following:

1. Create a **confusion matrix** and display the results (Hint: Feel free to use the Scikit-Learn `confusion_matrix` function).
2. Based on your confusion matrix, which digits seem to get confused with other digits the most?
 - 4 and 9 get misclassified as each other and 5 gets misclassified as 4

```

In [16]: from sklearn.metrics import confusion_matrix
# use your KNN class to perform KNN on the validation data with K = 3
knn = KNN(data.train_x, data.train_y, K=3)
val_yhat = knn.predict(data.val_x)

# create a confusion matrix
cm = confusion_matrix(data.val_y, val_yhat)

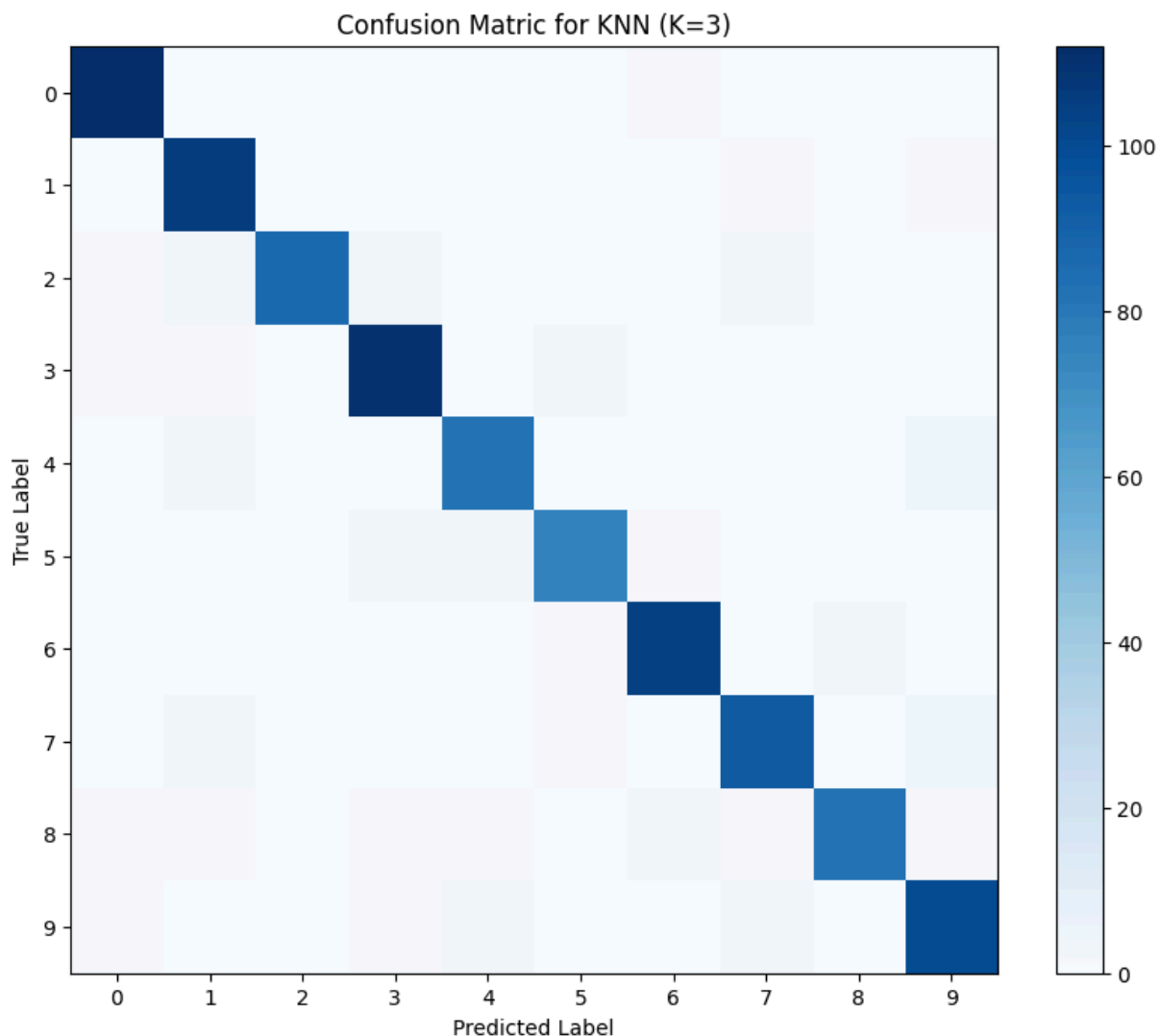
plt.figure(figsize=(10,8))
plt.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
plt.title("Confusion Matrix for KNN (K=3)")
plt.colorbar()

tick_marks = np.arange(10)
plt.xticks(tick_marks, tick_marks)
plt.yticks(tick_marks, tick_marks)

```

```
plt.xlabel("Predicted Label")
plt.ylabel("True Label")

plt.show()
```



Part 1.D [10 pts] Accuracy Plot:

1. Create a plot of the accuracy of the KNN on the test set on the same set of axes for $K=1,2,\dots,20$ (feel free to go out to $K=30$ if your implementation is efficient enough to allow it).
2. Based on the plot, which value of K results in highest accuracy?

```
In [17]: acc = []
wacc = []
allks = range(1,30)

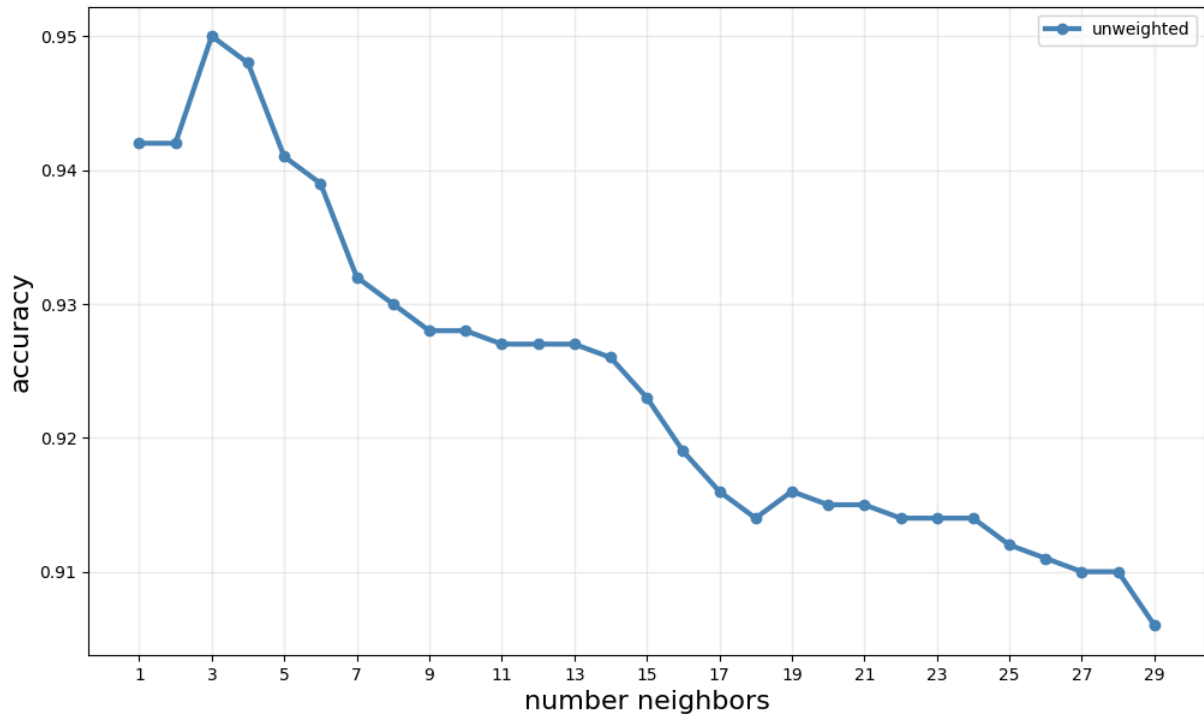
for k in allks:
    knn = KNN(data.train_x, data.train_y, K=k)
    y_pred = knn.predict(data.val_x)
```

```

accuracy = np.mean(y_pred == data.val_y)
acc.append(accuracy)

# you can use this code to create your plot
fig, ax = plt.subplots(nrows=1,ncols=1,figsize=(12,7))
ax.plot(allks, acc, marker="o", color="steelblue", lw=3, label="unweighted")
ax.set_xlabel("number neighbors", fontsize=16)
ax.set_ylabel("accuracy", fontsize=16)
plt.xticks(range(1,31,2))
ax.grid(alpha=0.25)
ax.legend()
plt.show()

```



Problem 2: Decision Tree, post-pruning and cost complexity parameter using sklearn > 0.22 [50 points]

We will use a pre-processed natural language dataset in the CSV file "spamdata.csv" to classify emails as spam or not. Each row contains the word frequency for 54 words plus statistics on the longest "run" of capital letters.

Word frequency is given by:

$$f_i = m_i / N$$

Where f_i is the frequency for word i , m_i is the number of times word i appears in the email, and N is the total number of words in the email.

We will use decision trees to classify the emails.

Part 2.A [5 points]: Complete the function `get_spam_dataset` to read in values from the dataset and split the data into train and test sets.

```
In [34]: def get_spam_dataset(filepath, test_split=0.1):
    """
    get_spam_dataset

    Loads csv file located at "filepath". Shuffles the data and splits
    it so that the you have (1-test_split)*100% training examples and
    (test_split)*100% testing examples.

    Args:
        filepath: location of the csv file
        test_split: percentage/100 of the data should be the testing split

    Returns:
        X_train, X_test, y_train, y_test, feature_names

        (in that order)
        first four are np.ndarray

    """

    df = pd.read_csv(filepath, delim_whitespace=True, compression='gzip')

    # checking why array are showing as 0 features
    #print("Columns in df:", df.columns)
    #print("Shape of df:", df.shape)
    #print(df.head())

    y = df.iloc[:, -1].values
    X = df.iloc[:, :-1].values
    feature_names = list(df.columns[:-1])

    #split into train/test sets
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_split, shuffle=True, random_state=42
    )

    return X_train, X_test, y_train, y_test, feature_names

test_split = 0.1 # default test_split; change it if you'd like; ensure that

X_train, X_test, y_train, y_test, label_names = get_spam_dataset(filepath="c
```

```
/var/folders/3k/dwgmpl79529bv_2x3vkrv60r0000gn/T/ipykernel_63731/2356137580.
py:21: FutureWarning: The 'delim_whitespace' keyword in pd.read_csv is depre
cated and will be removed in a future version. Use ``sep='\s+'`` instead
    df = pd.read_csv(filepath, delim_whitespace=True, compression='gzip')
```

```
In [ ]: # checking which delimiter is used
with gzip.open("data/spamdata.csv.gz", 'rt') as f:
    for _ in range(5):
        print(f.readline())
```

word_freq_make word_freq_address word_freq_all word_freq_3d word_freq_our word_freq_over word_freq_remove word_freq_internet word_freq_order word_freq_mail word_freq_receive word_freq_will word_freq_people word_freq_report word_freq_addresses word_freq_free word_freq_business word_freq_email word_freq_you word_freq_credit word_freq_your word_freq_font word_freq_000 word_freq_money word_freq_hp word_freq_hpl word_freq_george word_freq_650 word_freq_lab word_freq_labs word_freq_telnet word_freq_857 word_freq_data word_freq_415 word_freq_85 word_freq_technology word_freq_1999 word_freq_parts word_freq_pm word_freq_direct word_freq_cs word_freq_meeting word_freq_original word_freq_project word_freq_re word_freq_edu word_freq_table word_freq_conference char_freq_ ; char_freq_ (char_freq_ [char_freq_ ! char_freq_ \$ char_freq_ # capital_run_length_average capital_run_length_longest capital_run_length_total isSPAM

0 0.64 0.64 0 0.32 0 0 0 0 0 0 0.64 0 0 0 0.32 0 1.29 1.93 0 0.96 0 0 0 0 0
0 0.778 0 0 3.756 61 278 1

0.21 0.28 0.5 0 0.14 0.28 0.21 0.07 0 0.94 0.21 0.79 0.65 0.21 0.14 0.14 0.07
0.28 3.47 0 1.59 0 0.43 0.43 0 0 0 0 0 0 0 0 0 0 0.07 0 0 0 0 0 0 0 0
0 0 0 0 0.132 0 0.372 0.18 0.048 5.114 101 1028 1

0.06 0 0.71 0 1.23 0.19 0.19 0.12 0.64 0.25 0.38 0.45 0.12 0 1.75 0.06 0.06
1.03 1.36 0.32 0.51 0 1.16 0.06 0 0 0 0 0 0 0 0 0 0 0.06 0 0 0.12
0 0.06 0.06 0 0 0.01 0.143 0 0.276 0.184 0.01 9.821 485 2259 1

0 0 0 0 0.63 0 0.31 0.63 0.31 0.63 0.31 0.31 0.31 0 0 0.31 0 0 3.18 0 0.31 0
0 0.137 0 0.137 0 0 3.53
7 40 191 1

Part 2.B [5 points] : Build a decision tree classifier using the sklearn toolbox. Then compute metrics for performance like precision and recall. This is a binary classification problem, therefore we can label all points as either positive (SPAM) or negative (NOT SPAM).

```
In [36]: from sklearn.metrics import classification_report, precision_score, recall_score

def build_dt(data_X, data_y, max_depth = None, max_leaf_nodes =None):
    """
    This function builds the decision tree classifier and
    fits it to the provided data.

    Arguments
        data_X - a np.ndarray
        data_y - np.ndarray
        max_depth - None if unrestricted, otherwise an integer for the maximum
                    depth the tree can reach.

    Returns:
        A trained DecisionTreeClassifier
    """

    clf = DecisionTreeClassifier(max_depth=max_depth, max_leaf_nodes=max_leaf_nodes)
    clf.fit(data_X, data_y)
    return clf
```

```

# train model
dt_model = build_dt(X_train, y_train)

# predict on test set
y_pred = dt_model.predict(X_test)

# print metrics
print(classification_report(y_test, y_pred, target_names=["NOT SPAM", "SPAM"])

# individual metrics
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)

print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"Accuracy: {accuracy:.4f}")

```

	precision	recall	f1-score	support
NOT SPAM	0.90	0.97	0.94	266
SPAM	0.96	0.85	0.90	195
accuracy			0.92	461
macro avg	0.93	0.91	0.92	461
weighted avg	0.92	0.92	0.92	461

Precision: 0.9595

Recall: 0.8513

Accuracy: 0.9219

Part 2.C [10 pts]: Here we are going to use `calculate_precision` and `calculate_recall` functions to see how these metrics change when parameters of the tree are changed.

```

In [37]: def calculate_precision(y_true, y_pred, pos_label_value=1.0):
    """
    This function accepts the labels and the predictions, then
    calculates precision for a binary classifier.

    Args
    y_true: np.ndarray
    y_pred: np.ndarray

    pos_label_value: (float) the number which represents the positive
    label in the y_true and y_pred arrays. Other numbers will be taken
    to be the non-positive class for the binary classifier.

    Returns precision as a floating point number between 0.0 and 1.0
    """
    y_true = np.array(y_true)
    y_pred = np.array(y_pred)

    tp = np.sum((y_pred == pos_label_value) & (y_true == pos_label_value))

```

```

fp = np.sum((y_pred == pos_label_value) & (y_true != pos_label_value))

if tp + fp == 0:
    return 0

return tp / (tp + fp)

def calculate_recall(y_true, y_pred, pos_label_value=1.0):
    """
    This function accepts the labels and the predictions, then
    calculates recall for a binary classifier.

    Args
        y_true: np.ndarray
        y_pred: np.ndarray

        pos_label_value: (float) the number which represents the positive
        label in the y_true and y_pred arrays. Other numbers will be taken
        to be the non-positive class for the binary classifier.

    Returns precision as a floating point number between 0.0 and 1.0
    """

    y_true = np.array(y_true)
    y_pred = np.array(y_pred)

    tp = np.sum((y_pred == pos_label_value) & (y_true == pos_label_value))
    fn = np.sum((y_pred != pos_label_value) & (y_true == pos_label_value))

    if tp + fn == 0:
        return 0

    return tp / (tp + fn)

```

Part 2.D-1 [5 pts]: Modifying `max_depth` : - Create a model with a shallow `max_depth` of 2. Build the model on the training set. - Report precision/recall on the test set. - Report depth of the tree.

```

In [38]: # TODO : Complete the first subtask for max_depth
# precision =
# recall =

# build model with shallow depth
dt = build_dt(X_train, y_train, max_depth=2)

# predict on test set
y_pred = dt.predict(X_test)

# calculate precision and recall
precision = calculate_precision(y_test, y_pred)
recall = calculate_recall(y_test, y_pred)

print("Precision: {:.2f} Recall: {:.2f} Tree Depth: {}".format(precision,

```

Precision: 0.94 Recall: 0.67 Tree Depth: 2

Part 2.D-2 [5 pts]: Modifying `max_leaf_nodes` : - Create a model with a shallow `max_leaf_nodes` of 4. Build the model on the training set. - Report precision/recall on the test set. - Report depth of the tree.

```
In [39]: # TODO : Complete the second subtask for max_depth
# precision =
# recall =

# build model with shallow number of leaf nodes
dt = build_dt(X_train, y_train, max_leaf_nodes=4)

# predict on test set
y_pred = dt.predict(X_test)

# calculate precision and recall
precision = calculate_precision(y_test, y_pred)
recall = calculate_recall(y_test, y_pred)

print("Precision: {:.2f} Recall: {:.2f} No. of leaves: {}".format(precision,
```

Precision: 0.84 Recall: 0.82 No. of leaves: 4

Part 2.D-3 [10 pts]: Answer the following question:

How do precision and recall compare when you modify the max depth compared to the max number of leaf nodes? (Make sure to run your models a few times to get an idea).

Precision drops if the tree predicts more positives but more of them are false (more false positives) and recall drops if the tree misses actual positives (more false negatives). When you limit the depth, you may stop the tree from reaching conditions that capture edge cases so recall drops. When you limit the leaf nodes, the tree may still find useful splits without going deep, so recall is sometimes better but precision is worse.

Part 2.E [10 pts] : In class, we used `gridsearchCV` to do hyperparameter tuning to select the different parameters like `max_depth` to see how our tree grows and avoids overfitting. Here, we will use cost complexity pruning parameter α sklearn 0.22.1[https://scikit-learn.org/stable/user_guide.html] (or a later version) to prune our tree after training so as to improve accuracy on unseen data. In this exercise you will iterate over different `ccp_alpha` values and identify how performance is modulated by this parameter.

```
In [40]: dt = build_dt(X_train, y_train)

path = dt.cost_complexity_pruning_path(X_train, y_train) #post pruning
ccp_alphas, impurities = path.ccp_alphas, path.impurities

clfs = [] # VECTOR CONTAINING CLASSIFIERS FOR DIFFERENT ALPHAS
# TODO: iterate over ccp_alpha values
for alpha in ccp_alphas:
    clf = DecisionTreeClassifier(random_state=42, ccp_alpha=alpha)
    clf.fit(X_train, y_train)
```

```

        clfs.append(clf)

print("Number of nodes in the last tree is: {} with ccp_alpha: {}".format(
    clfs[-1].tree_.node_count, ccp_alphas[-1]))

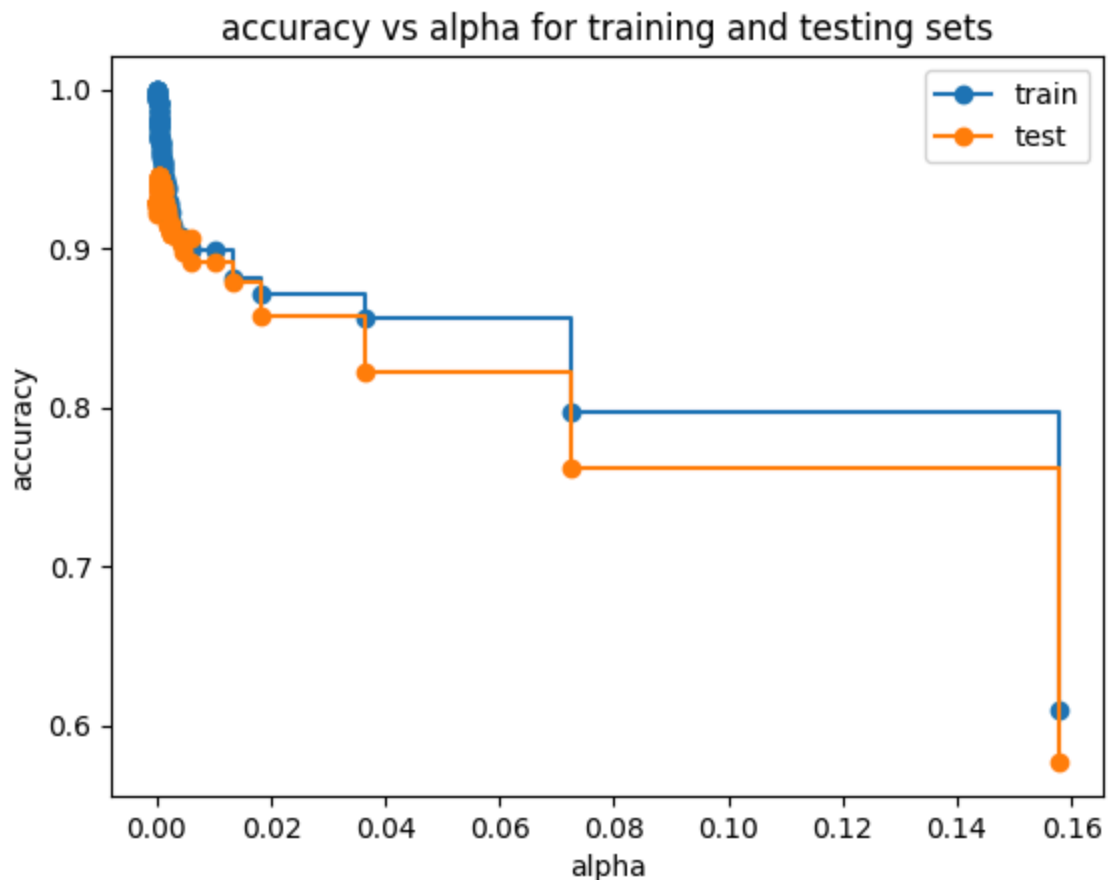
# TODO: next, generate the train and test scores and plot the variation in t
# The code for plotting has been provided; edit the train_scores and test_sc
train_scores = []
test_scores = []

for clf in clfs:
    train_acc = clf.score(X_train, y_train)
    test_acc = clf.score(X_test, y_test)
    train_scores.append(train_acc)
    test_scores.append(test_acc)

fig, ax = plt.subplots()
ax.set_xlabel("alpha")
ax.set_ylabel("accuracy")
ax.set_title("accuracy vs alpha for training and testing sets")
ax.plot(ccp_alphas, train_scores, marker='o', label="train",
        drawstyle="steps-post")
ax.plot(ccp_alphas, test_scores, marker='o', label="test",
        drawstyle="steps-post")
ax.legend()
plt.show()

```

Number of nodes in the last tree is: 1 with ccp_alpha: 0.15776315514960632



Submission

Make sure you have run all cells in your notebook in order before running the cell below, so that all images/graphs appear in the output. The cell below will generate a zip file for you to submit. **Please save before exporting!**

These are some submission instructions.

```
In [42]: # Save your notebook first, then run this cell to export your submission.  
grader.export(run_tests=True)
```

```

-----
PandocMissing                                     Traceback (most recent call last)
File ~/Library/Python/3.9/lib/python/site-packages/otter/export/exporters/via_latex.py:64, in PDFViaLatexExporter.convert_notebook(cls, nb_path, dest, xecjk, no_xecjk, **kwargs)
    63 try:
----> 64     pdf_output = pdf_exporter.from_notebook_node(nb)
    65     with open(dest, "wb") as output_file:

File ~/Library/Python/3.9/lib/python/site-packages/nbconvert/exporters/pdf.py:184, in PDFExporter.from_notebook_node(self, nb, resources, **kw)
    183 """Convert from notebook node."""
--> 184 latex, resources = super().from_notebook_node(nb, resources=resources, **kw)
    185 # set texinputs directory, so that local files will be found

File ~/Library/Python/3.9/lib/python/site-packages/nbconvert/exporters/latex.py:92, in LatexExporter.from_notebook_node(self, nb, resources, **kw)
    90 self.register_filter("convert_explicitly_relative_paths", convert_explicitly_relative_paths)
----> 92 return super().from_notebook_node(nb, resources, **kw)

File ~/Library/Python/3.9/lib/python/site-packages/nbconvert/exporters/templateexporter.py:429, in TemplateExporter.from_notebook_node(self, nb, resources, **kw)
    428 # Top level variables are passed to the template_exporter here.
--> 429 output = self.template.render(nb=nb_copy, resources=resources)
    430 output = output.lstrip("\r\n")

File ~/Library/Python/3.9/lib/python/site-packages/jinja2/environment.py:1295, in Template.render(self, *args, **kwargs)
    1294 except Exception:
--> 1295     self.environment.handle_exception()

File ~/Library/Python/3.9/lib/python/site-packages/jinja2/environment.py:942, in Environment.handle_exception(self, source)
    940 from .debug import rewrite_traceback_stack
--> 942 raise rewrite_traceback_stack(source=source)

File ~/Library/Python/3.9/lib/python/site-packages/otter/export/exporters/templates/via_latex/index.tex.j2:7, in top-level template code()
     6 % Inherit from the specified cell style.
----> 7 ((* extends cell_style *))
     8

File ~/Library/Python/3.9/share/jupyter/nbconvert/templates/latex/style_ipython.tex.j2:68, in top-level template code()
    67 \begin{Verbatim}[commandchars=\\\{\}]
----> 68 ((( text | add_prompts(first='{\color{' ~ prompt_color ~ '}' ~ prompt ~ '[{\color{' ~ prompt_color ~ '}' ~ execution_count ~ '}]:' ~ ', cont=indentation) )))
    69 \end{Verbatim}

File ~/Library/Python/3.9/share/jupyter/nbconvert/templates/latex/base.tex.j2:7, in top-level template code()
     6

```

```

----> 7 ((* extends 'document_contents.tex.j2' *))
      8

```

File ~/Library/Python/3.9/share/jupyter/nbconvert/templates/latex/document_contents.tex.j2:51, in top-level template code()

```

      50 ((* set filename = filename | posix_path *))
----> 51 ((* block figure scoped *))
      52     \begin{center}

```

File ~/Library/Python/3.9/share/jupyter/nbconvert/templates/latex/display_priority.j2:5, in top-level template code()

```

      4
----> 5 ((* extends 'null.j2' *))
      6

```

File ~/Library/Python/3.9/share/jupyter/nbconvert/templates/latex/null.j2:30, in top-level template code()

```

      29 ((* endblock header *))
----> 30 ((* block body *))
      31 ((* for cell in nb.cells *))

```

File ~/Library/Python/3.9/share/jupyter/nbconvert/templates/latex/base.tex.j2:241, in block 'body'()

```

      240
--> 241     ((( super() )))
      242

```

File ~/Library/Python/3.9/share/jupyter/nbconvert/templates/latex/null.j2:32, in block 'body'()

```

      31 ((* for cell in nb.cells *))
----> 32     ((* block any_cell scoped *))
      33         ((* if cell.cell_type == 'code' *))

```

File ~/Library/Python/3.9/share/jupyter/nbconvert/templates/latex/null.j2:85, in block 'any_cell'()

```

      84 ((* if resources.global_content_filter.include_markdown and not cell.metadata.get("transient", {}).get("remove_source", false) *))
----> 85     ((* block markdowncell scoped *)) ((* endblock markdowncell *))
      86 ((* endif *))

```

File ~/Library/Python/3.9/lib/python/site-packages/otter/export/exporters/templates/via_latex/index.tex.j2:32, in block 'markdowncell'()

```

      31         ((* endif *))
----> 32     ((( super() )))
      33 ((* endblock markdowncell *))

```

File ~/Library/Python/3.9/share/jupyter/nbconvert/templates/latex/document_contents.tex.j2:68, in block 'markdowncell'()

```

      67 ((* block markdowncell scoped *))
----> 68     ((( cell.source | citation2latex | strip_files_prefix | convert_pandoc('markdown+tex_math_double_backslash', 'json', extra_args=[]) | resolve_references | convert_explicitly_relative_paths | convert_pandoc('json', 'latex') )))
      69 ((* endblock markdowncell *))

```

```

File ~/Library/Python/3.9/lib/python/site-packages/nbconvert/filters/pandoc.py:36, in convert_pandoc(source, from_format, to_format, extra_args)
    17 """Convert between any two formats using pandoc.
    18
    19 This function will raise an error if pandoc is not installed.
    (...)
    34 Output as returned by pandoc.
    35 """
--> 36 return pandoc(source, from_format, to_format, extra_args=extra_args)

```

```

File ~/Library/Python/3.9/lib/python/site-packages/nbconvert/utils/pandoc.py:50, in pandoc(source, fmt, to, extra_args, encoding)
    49 # this will raise an exception that will pop us out of here
--> 50 check_pandoc_version()
    52 # we can safely continue

```

```

File ~/Library/Python/3.9/lib/python/site-packages/nbconvert/utils/pandoc.py:98, in check_pandoc_version()
    96 return check_pandoc_version._cached # type:ignore[attr-defined]
--> 98 v = get_pandoc_version()
    99 if v is None:

```

```

File ~/Library/Python/3.9/lib/python/site-packages/nbconvert/utils/pandoc.py:75, in get_pandoc_version()
    74 if not shutil.which("pandoc"):
--> 75     raise PandocMissing()
    77 out = subprocess.check_output(["pandoc", "-v"]) # noqa: S607, S603

```

PandocMissing: Pandoc wasn't found.
Please check that pandoc is installed:
<https://pandoc.org/installing.html>

During handling of the above exception, another exception occurred:

```

AttributeError                                Traceback (most recent call last)
Cell In[42], line 2
      1 # Save your notebook first, then run this cell to export your submission.
--> 2 grader.export(run_tests=True)

```

```

File ~/Library/Python/3.9/lib/python/site-packages/otter/check/utils.py:97, in colab_incompatible.<locals>.colab_only_method(self, *args, **kwargs)
    95 if self._colab:
    96     raise RuntimeError("This method is not compatible with Google Colab")
--> 97 return f(self, *args, **kwargs)

```

```

File ~/Library/Python/3.9/lib/python/site-packages/otter/check/utils.py:131, in logs_event.<locals>.event_logger.<locals>.run_function(self, *args, **kwargs)
    129 except Exception as e:
    130     self._log_event(event_type, success=False, error=e)
--> 131     raise e
    132 else:
    133     self._log_event(event_type, results=results, question=question,
shelve_env=shelve_env)

```

```
File ~/Library/Python/3.9/lib/python/site-packages/otter/check/utils.py:126,
in logs_event.<locals>.event_logger.<locals>.run_function(self, *args, **kwargs)
124     question, results, shelve_env = f(self, *args, **kwargs)
125 else:
```

```
--> 126     results = f(self, *args, **kwargs)
127     shelve_env = {}
128     question = None
```

```
File ~/Library/Python/3.9/lib/python/site-packages/otter/check/notebook.py:3
24, in Notebook.export(self, nb_path, export_path, pdf, filtering, pagebreak
s, files, display_link, force_save, run_tests)
```

```
322 pdf_path = ".".join(nb_path.split(".")[:-1]) + ".pdf"
323 # convert(nb_path, filtering=filtering, filter_type=filter_type)
--> 324 export_notebook(nb_path, filtering=filtering, pagebreaks=pagebreaks)
325 if os.path.isfile(pdf_path):
326     zf.write(pdf_path)
```

```
File ~/Library/Python/3.9/lib/python/site-packages/otter/export/__init__.py:
31, in export_notebook(nb_path, dest, exporter_type, **kwargs)
```

```
28 pdf_name = os.path.splitext(nb_path)[0] + ".pdf"
30 Exporter = get_exporter(exporter_type=exporter_type)
--> 31 Exporter.convert_notebook(nb_path, pdf_name, **kwargs)
```

```
File ~/Library/Python/3.9/lib/python/site-packages/otter/export/exporters/vi
a_latex.py:78, in PDFViaLatexExporter.convert_notebook(cls, nb_path, dest, x
ecjk, no_xecjk, **kwargs)
```

```
75 if NB_CONVERT_6:
76     nbconvert.TemplateExporter.template_name = orig_template_nam
e
--> 78 except nbconvert.pdf.LatexFailed as error:
79     if not xecjk and not no_xecjk:
80         success = cls.convert_notebook(nb_path, dest, xecjk=True, **
kwargs)
```

```
AttributeError: module 'nbconvert' has no attribute 'pdf'
```